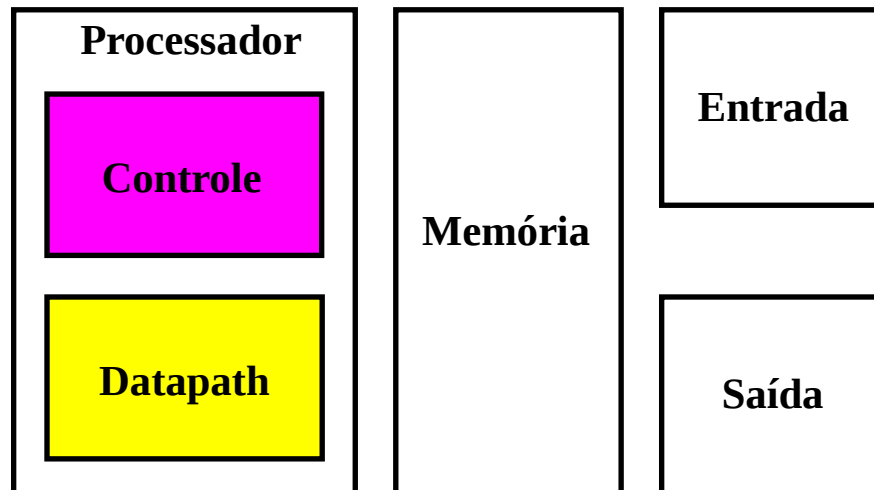


Pipelining avançado – Parte 2



Emissão múltipla dinâmica

- **Superscalar**
- **Instruções emitidas em ordem**
 - CPU decide número de instruções emitidas
 - » Nenhuma, uma ou mais
- **Compilador favorece**
 - Reordenando de instruções
 - Para aumentar chances de emissão múltipla

Emissão múltipla dinâmica

- **Ao migrar entre implementações, ...**
 - Código herdado (“legacy code”)
- **VLIW (geralmente) requer recompilação (o IA-64 não)**
 - Para garantir execução correta e/ou
 - Para assegurar desempenho aceitável
- **Superscalar garante execução correta**
 - Independentemente do código gerado
 - » Detecção de hazards e geração de pausas
- **Superscalar garante execução eficiente**
 - Mesmo sem recompilação
 - » Escalonamento dinâmico

Pipeline: escalonamento dinâmico

- **Motivação**

lw \$t0, 20(\$s2)

addu \$t1, \$t0, \$t2

sub \$s4, \$s4, \$t3

slti \$t5, \$s4, 20

Pode se revelar muito lenta em tempo de execução

Não depende de lw nem de addu, mas não pode iniciar execução antes delas.

A instrução sub poderia completar antes das duas primeiras

Isto requer **execução fora de ordem** (“out-of-order” execution)

Pipeline com escalonamento dinâmico

ILP: Especulação

- HW “adivinha” propriedade de instrução
 - Resultado do teste de um desvio condicional
 - Se um load depende de um store
- Para permitir sua **execução antecipada**
- Desvios
 - Executar uma instrução que sucede um desvio
 - **Antes** de ser conhecido o **resultado do teste**
- Load
 - Executar um load que sucede um store
 - **Antes** de saber se acessam o **mesmo endereço**

Especulação (dinâmica): desvio

if (A==0) A = B; else A = 4;

Hipótese: 0(\$s3)→ A; 0(\$s2)→B

Previsão: desvio tomado

(Teste em EX; atalho MEM->EX)

gera
hazard

lw \$s1, 0(\$s3)
↓
bne \$s1, \$zero, L1

lw \$s1, 0(\$s2)

j L2

L1: addi \$s1, \$zero, 4

L2: sw \$s1, 0(\$s3)

**Solução: armazenar o
resultado provisoriamente
em um buffer**

**Novo valor de \$s1 não
pode ser escrito no banco
de registradores**

lw ~~\$s1~~, 0(\$s3)

addi **\$s1'**, \$zero, 4

bne \$s1, \$zero, L1

lw \$s1, 0(\$s2)

j L2

L1: addi \$s1, \$zero, 4

L2: sw \$s1, 0(\$s3)

**Só atualizar o banco de
registradores (s1 = s1') se
hipótese resultar verdadeira**

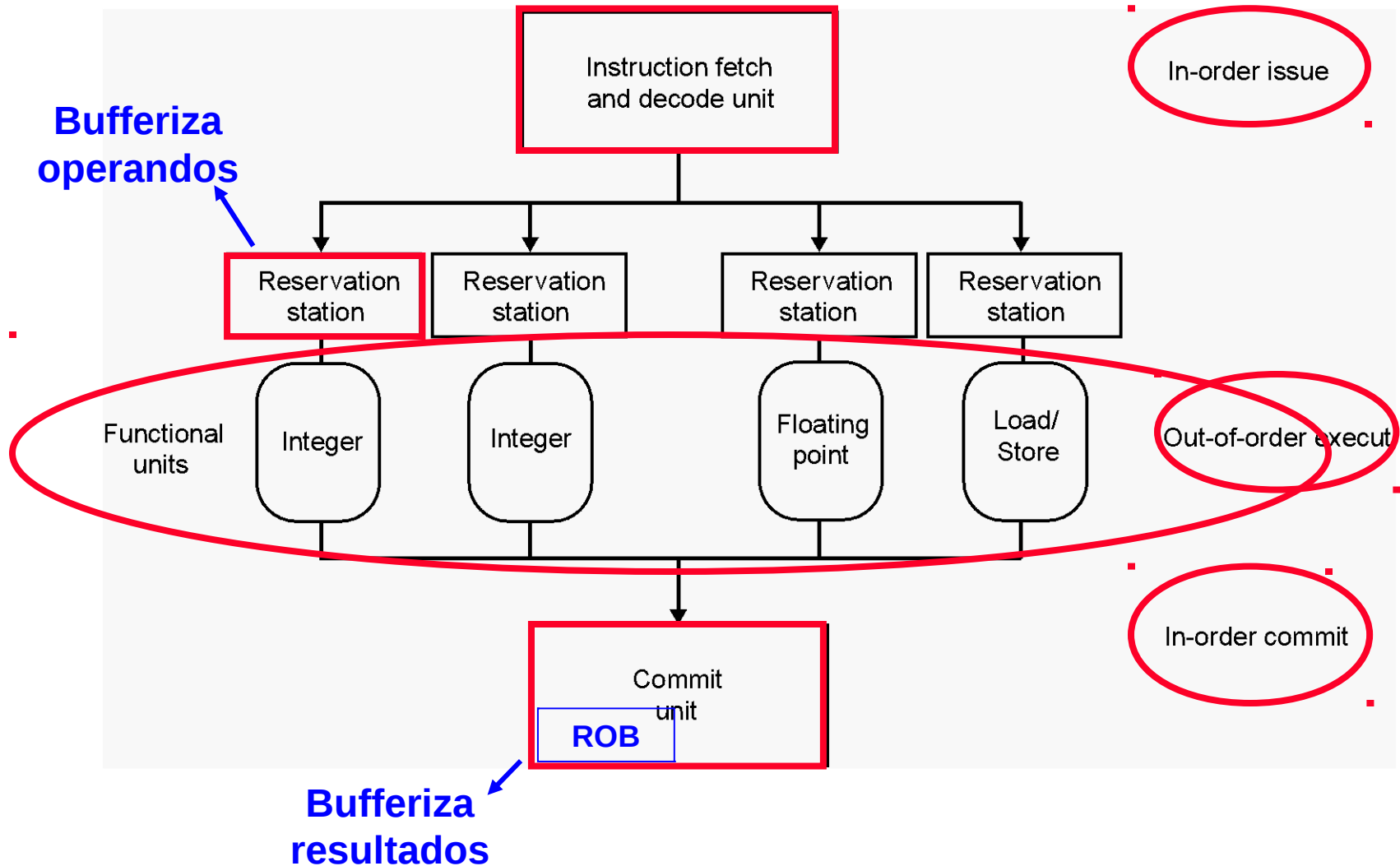
Especulação (dinâmica): load

```
sw    $s1, 0($s3)
...
lw    $s2, 4($s4)
```

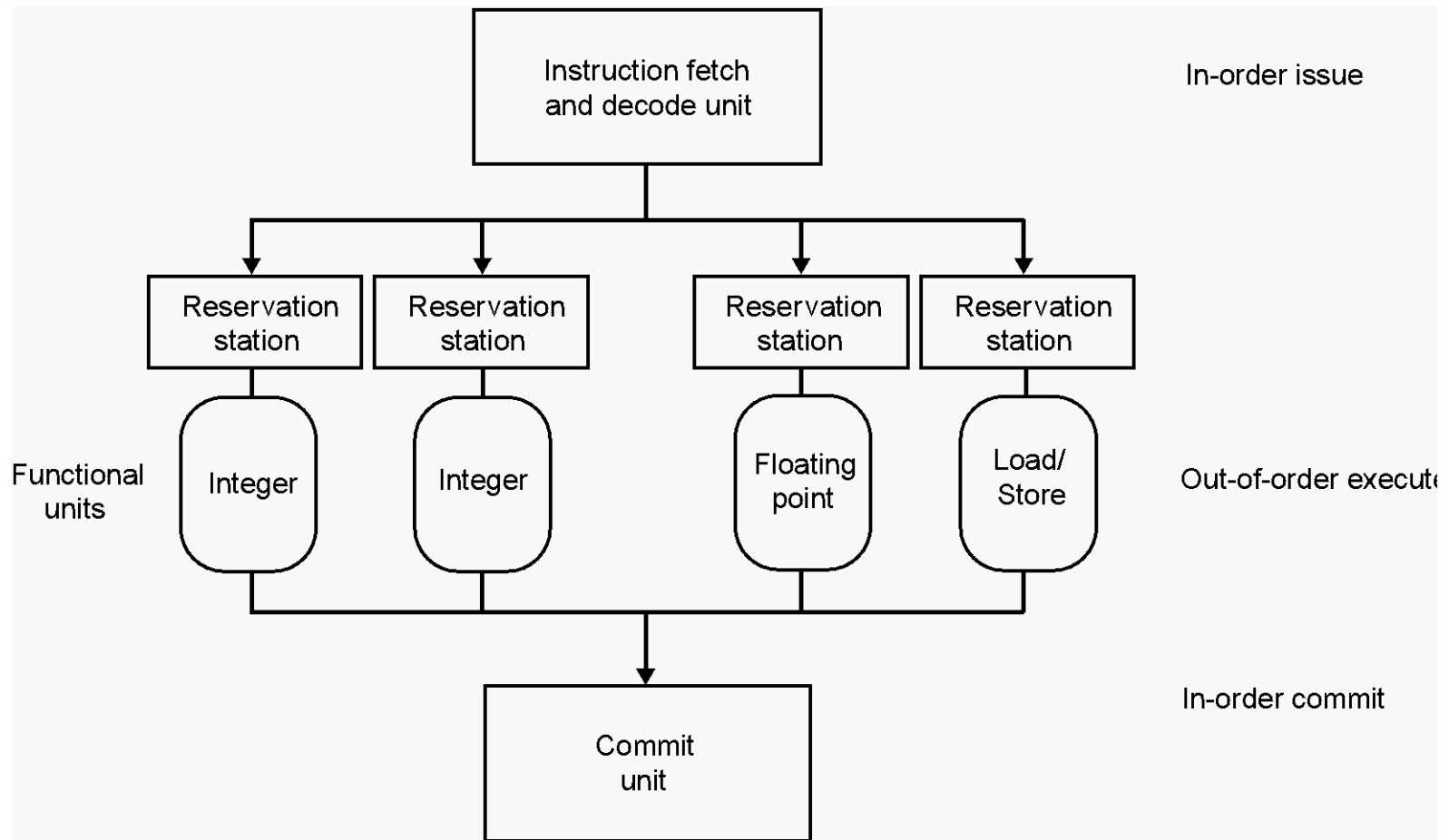


- Pode-se executar o lw antes do sw ?
 - Se $0 + \$s3 = 4 + \$s4$, são dependentes
 - » Não se pode reordená-las
- Pode-se fazê-lo desde que:
 - O valor lido pelo lw não seja escrito em \$s2
 - Seja armazenado em buffer até que...
 - Endereços efetivos sejam ambos conhecidos

Pipeline: escalonamento dinâmico



Pipeline: escalonamento dinâmico



Por que escalonamento dinâmico?

- Por que não deixar isso para o compilador?

Por que escalonamento dinâmico?

- **Reduz pausas não previsíveis estaticamente**
 - Isto é, aquelas não previsíveis **em tempo de compilação**
 - Exemplo: as geradas por faltas na cache
- **Absorve diferenças de latência entre implementações de uma mesma ISA**
 - Isto é, código compilado para Implementação 1
 - Roda eficientemente na Implementação 2
 - Consequências:
 - » Distribuidor de SW não precisa manter várias versões
 - » Usuário não precisa se preocupar com a versão
 - » Código velho roda rápido na máquina nova **sem recompilação!**

Eficiência energética

- **Escalonamento dinâmico e especulação**
 - Aumentam desempenho,
 - **Mas costumam ser energeticamente ineficientes**
- **Para obter mais desempenho por Joule,**
 - **Processadores tornaram-se menos sofisticados:**
 - » Pipelines menos profundos
 - » Especulação menos agressiva

Evolução dos pipelines

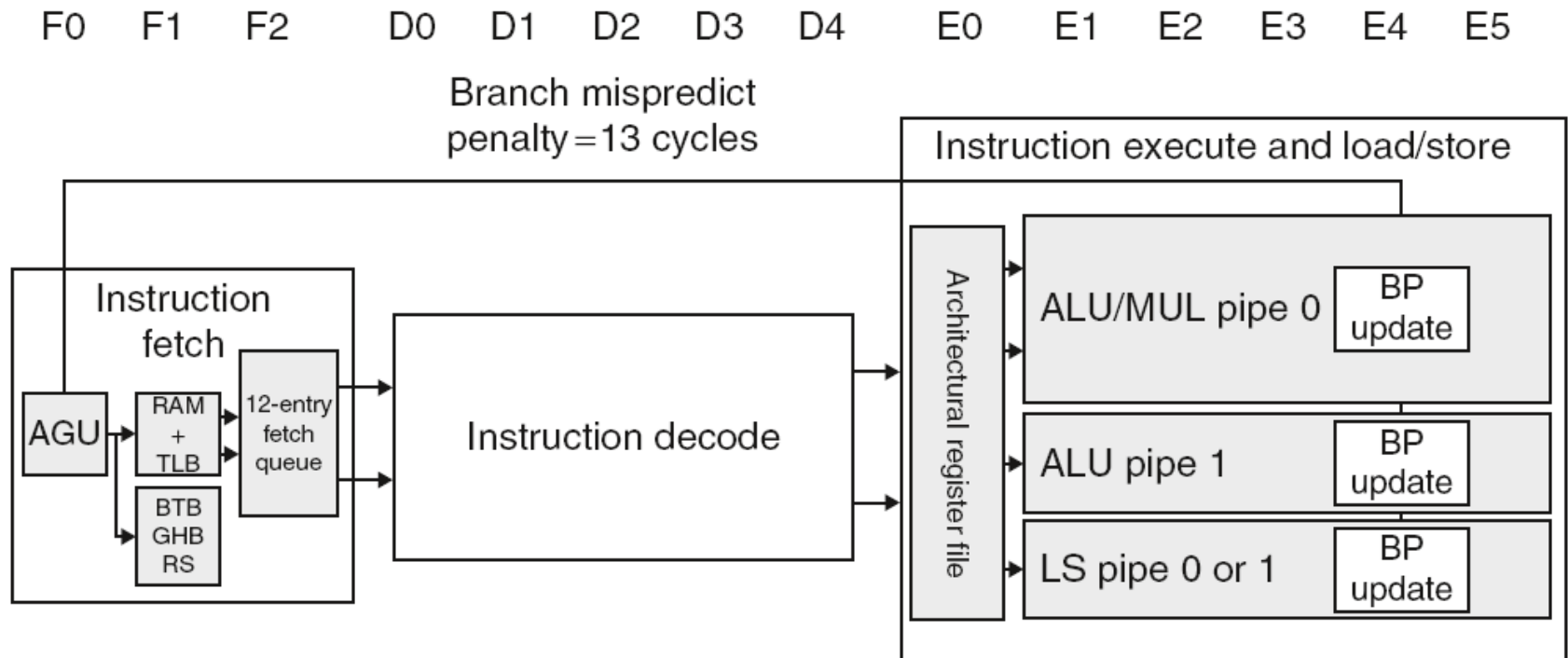
- **Redução do número de estágios e potência**
 - À medida que fabricantes foram mudando para multicores
- **Saturação da taxa de emissão**

Microprocessor	Year	Clock Rate	Pipeline Stages	Issue width	Out-of-order/ Speculation	Cores	Power
i486	1989	25MHz	5	1	No	1	5W
Pentium	1993	66MHz	5	2	No	1	10W
Pentium Pro	1997	200MHz	10	3	Yes	1	29W
P4 Willamette	2001	2000MHz	22	3	Yes	1	75W
P4 Prescott	2004	3600MHz	31	3	Yes	1	103W
Core	2006	2930MHz	14	4	Yes	2	75W
Core i5 Nehalem	2010	3300MHz	14	4	Yes	2-4	87W
Core i5 Ivy Bridge	2012	3400MHz	14	4	Yes	8	77W

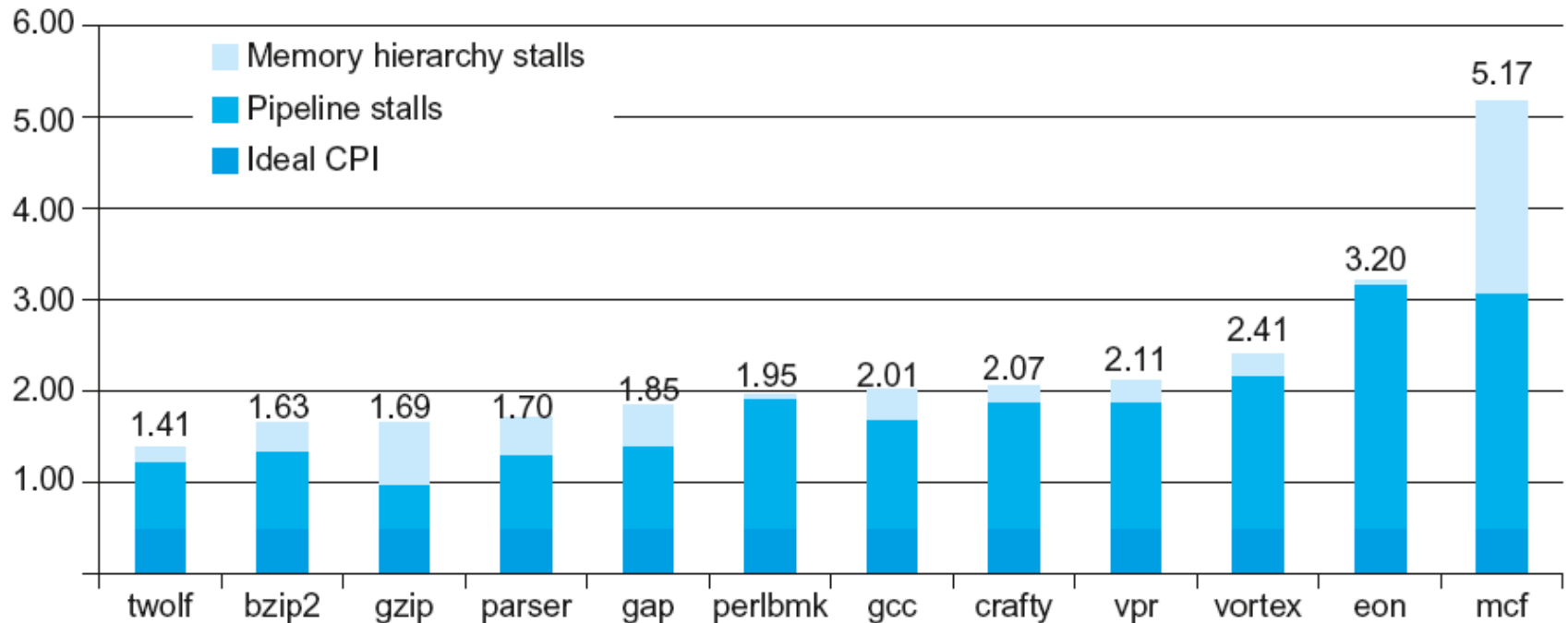
Comparação entre pipelines

Processor	ARM A8	Intel Core i7 920
Market	Personal Mobile Device	Server, cloud
Thermal design power	2 Watts	130 Watts
Clock rate	1 GHz	2.66 GHz
Cores/Chip	1	4
Floating point?	No	Yes
Multiple issue?	Dynamic	Dynamic
Peak instructions/clock cycle	2	4
Pipeline stages	14	14
Pipeline schedule	Static in-order	Dynamic out-of-order with speculation
Branch prediction	2-level	2-level
1 st level caches/core	32 KiB I, 32 KiB D	32 KiB I, 32 KiB D
2 nd level caches/core	128-1024 KiB	256 KiB
3 rd level caches (shared)	-	2- 8 MB

O pipeline do ARM Cortex-A8

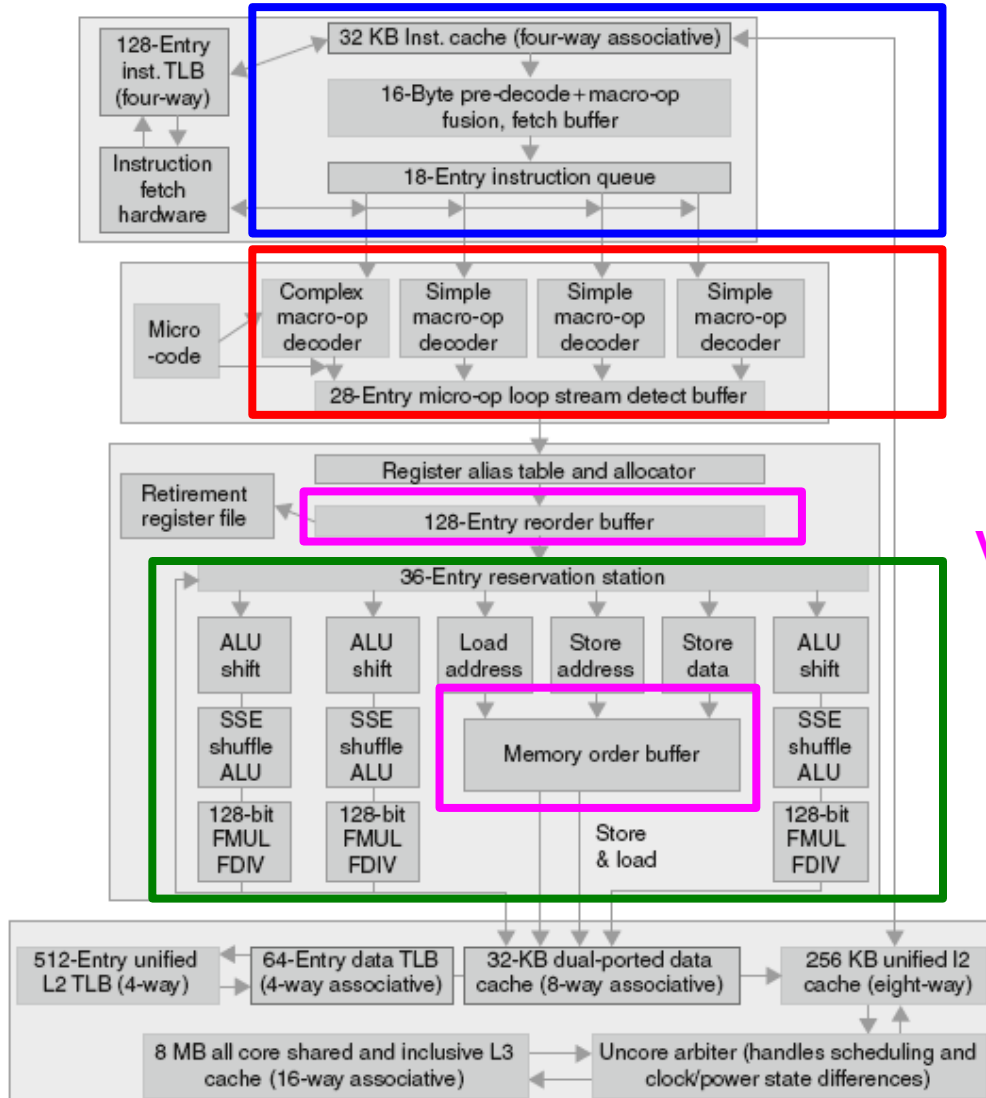


CPI para o ARM Cortex A8



- Valores bem acima do ideal (0,5)
- Pausas geradas no caso médio (gcc)
 - 80%: pipeline hazards, 20%: faltas no acesso a memória

O pipeline do Intel Core i7 920



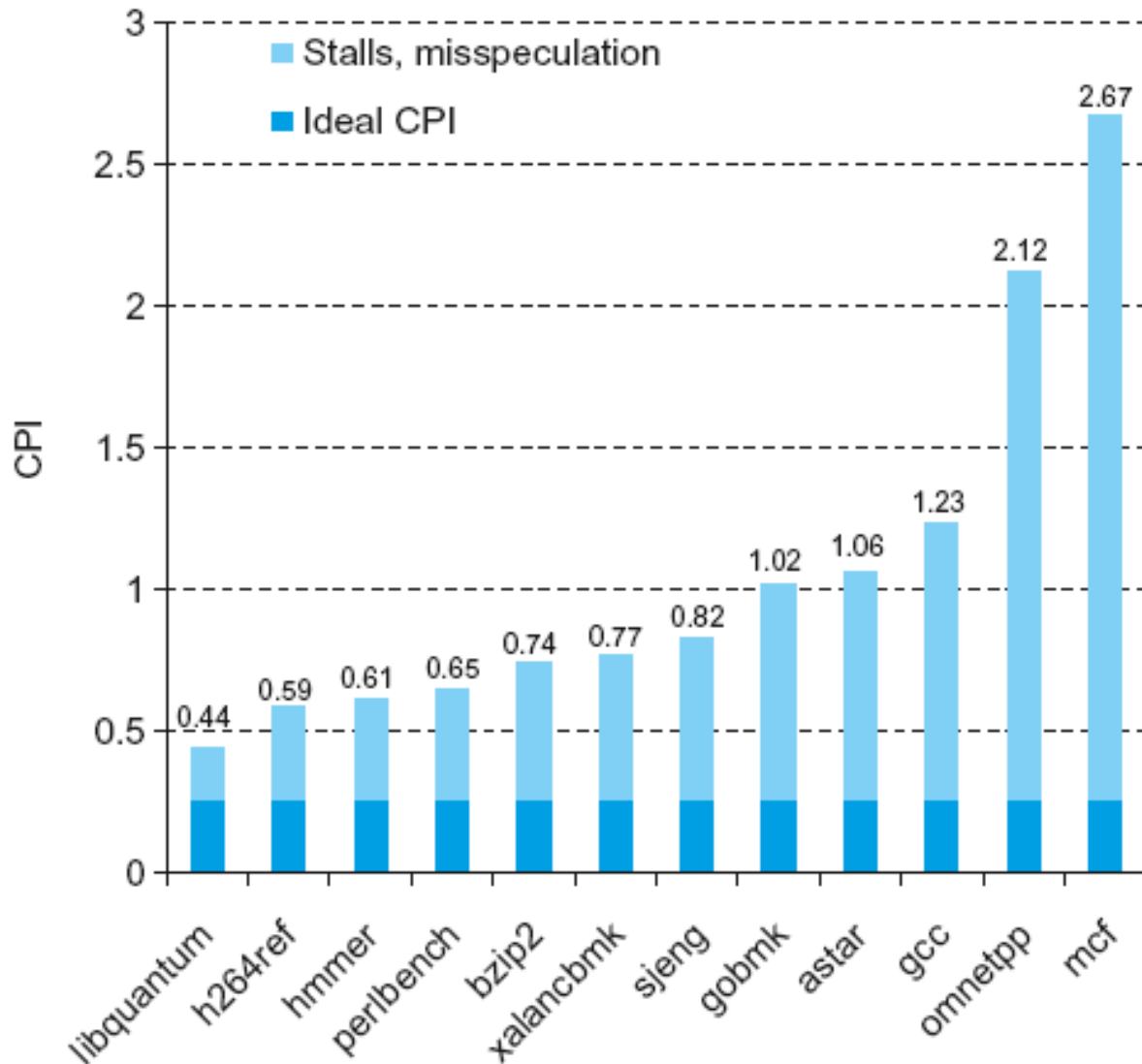
Busca e decodificação de instruções

Decomposição de instruções:
“CISC” em “RISC”

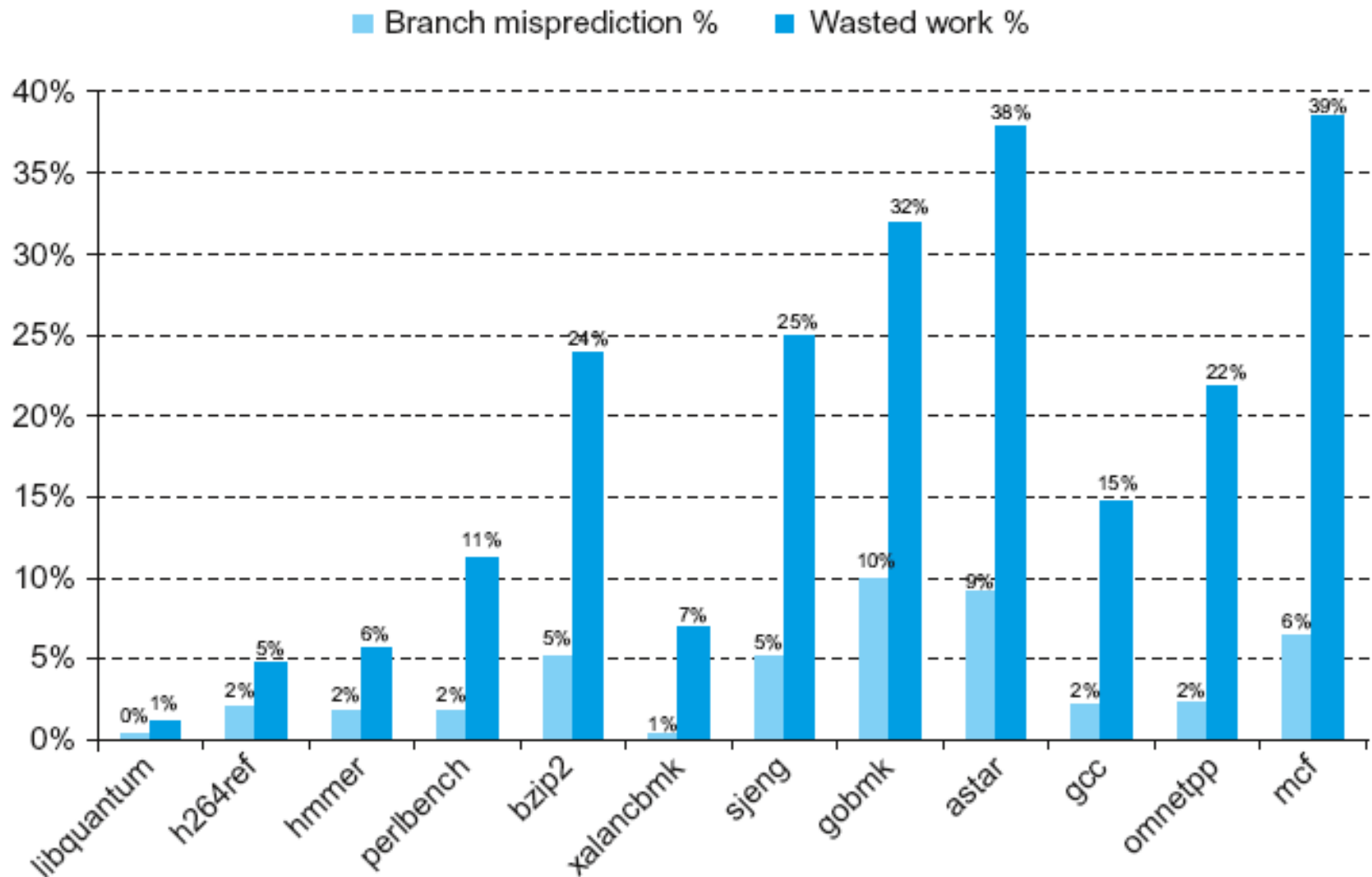
Validação ou descarte de resultado
 (“commit” ou “squash”)

Bufferização de operandos,
execução, bufferização de
resultados

CPI para o Intel Core i7 920



O impacto da má especulação no i7



Conclusões

Desempenho: ideias-chave até 2004

Miniaturização e superpipelining

$$IPS = \frac{f}{CPI}$$

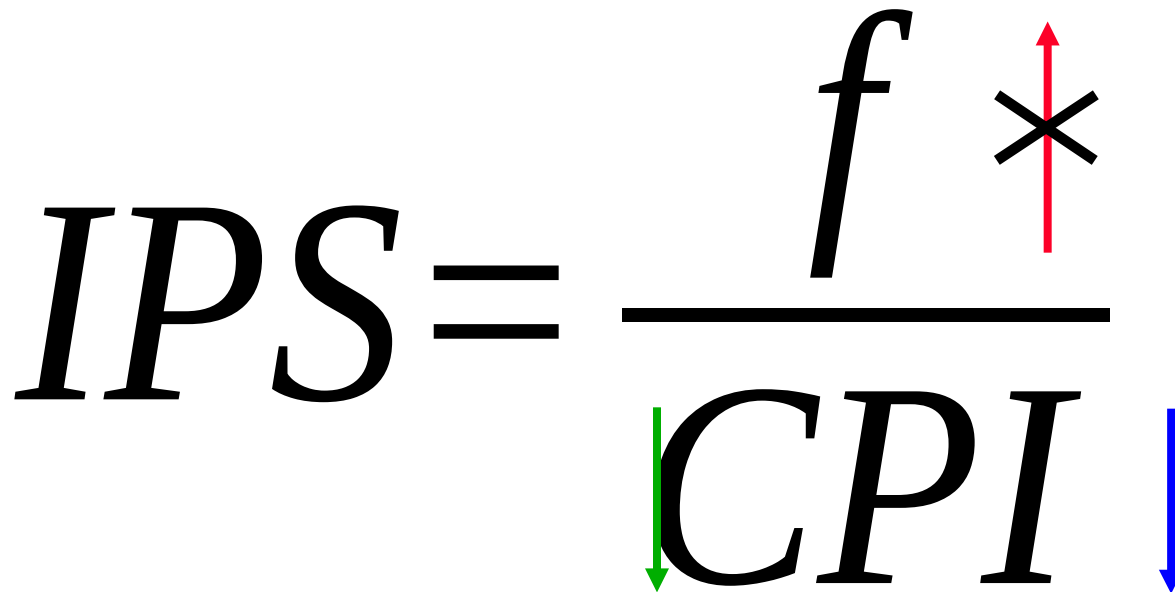

Emissão múltipla, escalonamento (dinâmico e estático),
especulação, previsão de desvios (estática e dinâmica),
memórias cache

Desempenho: ideias-chave até 2004

- Programador: desempenho sem esforço
 - Frequência de relógio crescia todo ano ($f \uparrow$)
 - Compilador paralelizava o programa ($CPI \downarrow$)
 - Escalonamento dinâmico compensava degradação do CPI devida a
 - » Não-recompilação para nova microarquitetura.
Exemplo: SW feito para latências da máquina antiga, acabava rodando eficientemente em máquina nova.
 - » Mal uso do compilador. Exemplo: mesmo usando-se flags default, SW acabava rodando eficientemente.
- Mas a realidade tem mudado desde então ...

Desempenho: ideias-chave após 2004

Miniaturização e ~~super~~pipelining

$$IPS = \frac{f}{CPI}$$


ILP: Emissão múltipla, escalonamento de instruções (dinâmico e estático), especulação, previsão de desvios (estática e dinâmica), memórias cache

TLP: Múltiplos cores, escalonamento de threads, modelos de consistência de memória relaxados